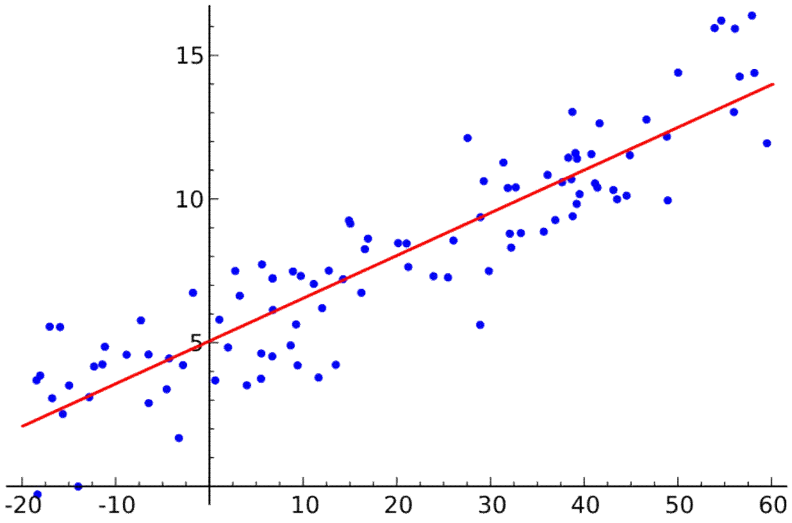
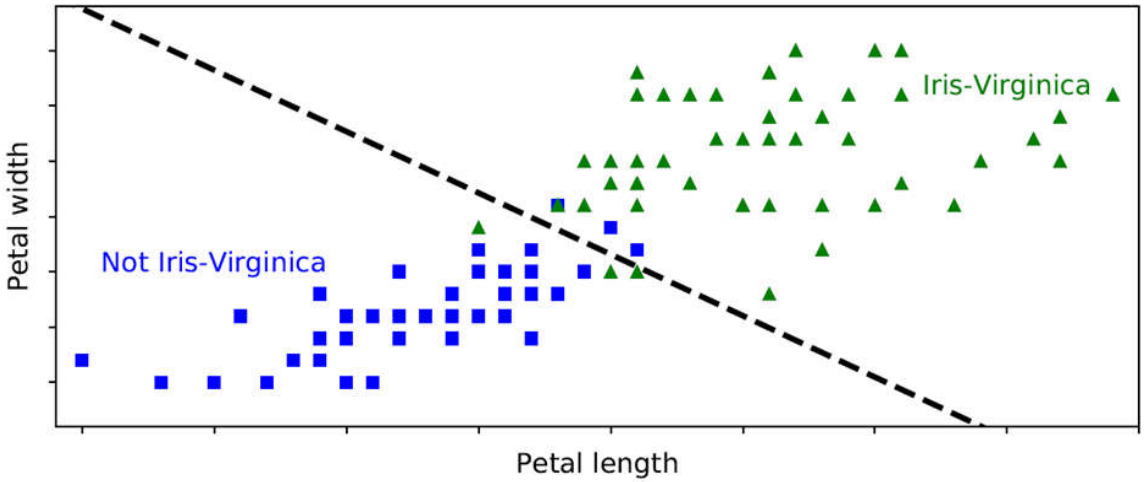
یادگیری نظارتی

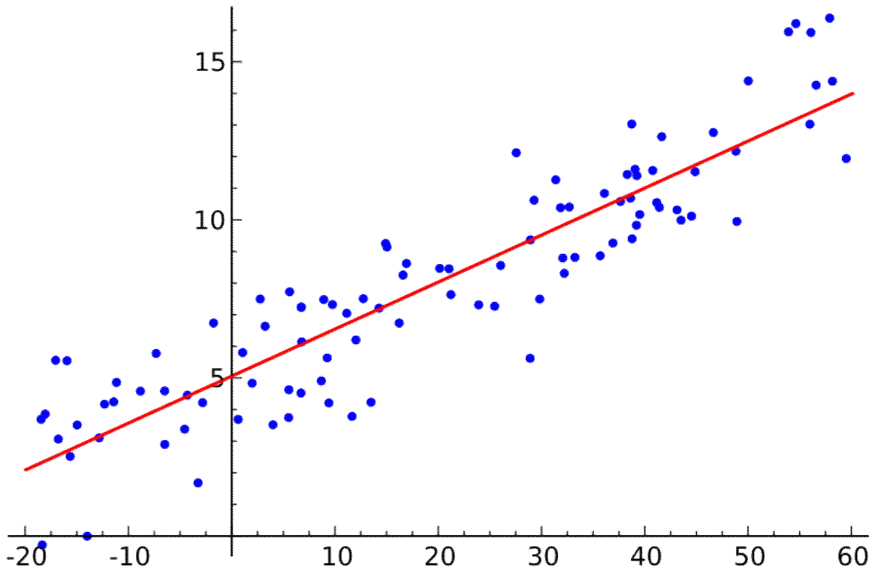
۱ پیش بینی یک متغیر پیوسته (Regression)

۲ تخمین یک متغیر گسسته (Classification)





رگرسیون(برازش)



انواع برازش

۱. برازش خطی ساده (Simple Linear Regression)

۲. برازش چندمتغیره (Multivariate Regression)

۳. برازش چندجمله‌ای (Polynomial Regression)

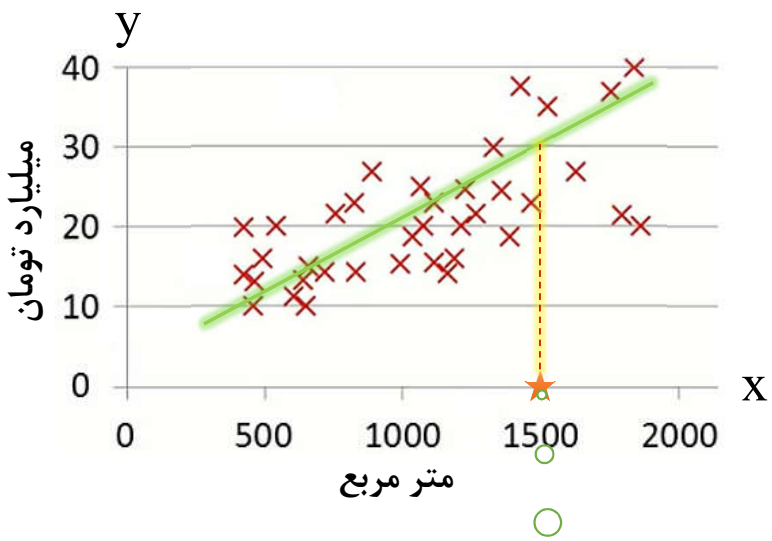
۴. و ...

برازش خطی ساده (Simple Linear Regression)

برازش خطی: ساده‌ترین روش برای یادگیری آماری است که رابطه میان یک متغیر وابسته با یک یا چند متغیر مستقل را مدل می‌کند.

مثال

برازش خطی با یک متغیر: پیش‌بینی قیمت خانه



x	y
۱۸۰۰	۲۰
۸۰۰	۱۳
۵۰۰	۱۲
۸۵۲	۱۴.۵
...	...

داده‌های آموزشی

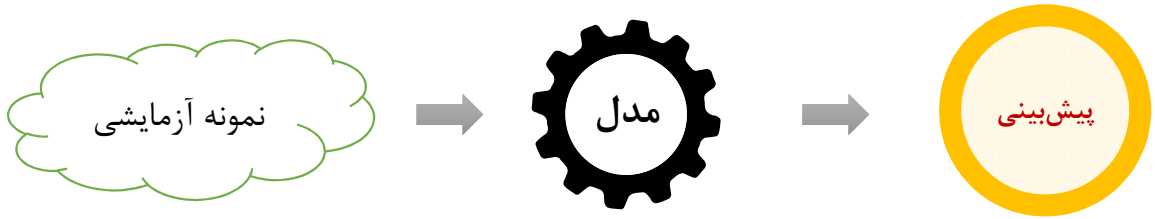
m: تعداد داده‌های آموزشی

x: متغیر ورودی / ویژگی‌ها / متغیر مستقل

y: متغیر خروجی/ متغیر هدف / متغیر پاسخ

$(x^{(i)}, y^{(i)})$: امین داده آموزشی i

(x, y): یک داده آموزشی



Simple Linear Regression

- we can write this linear relationship as:

$$Y \approx \theta_0 + \theta_1 X.$$

- For example, X may represent home size and Y may represent price.

Then we can regress price onto size by fitting the model

$$\text{price} \approx \theta_0 + \theta_1 \times \text{size}.$$

Training set of housing prices (Portland, OR)

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

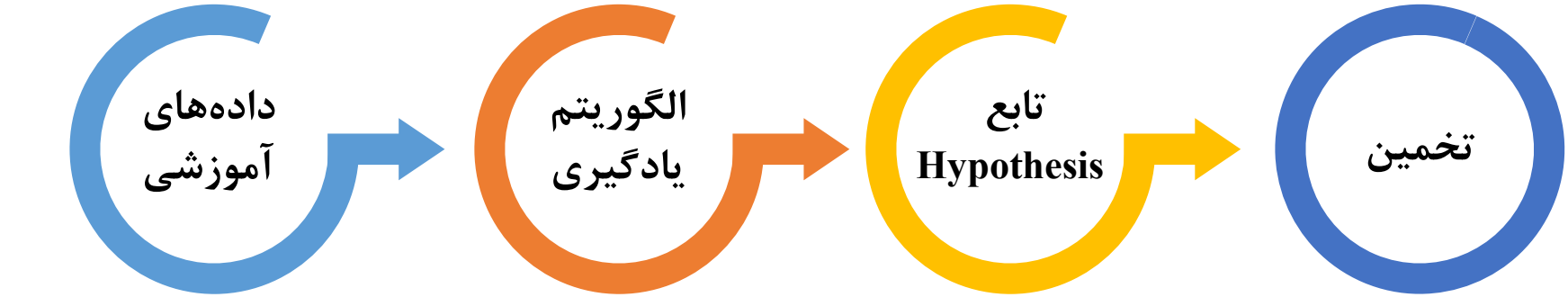
Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable

رگرسیون خطی ساده (Simple Linear Regression)

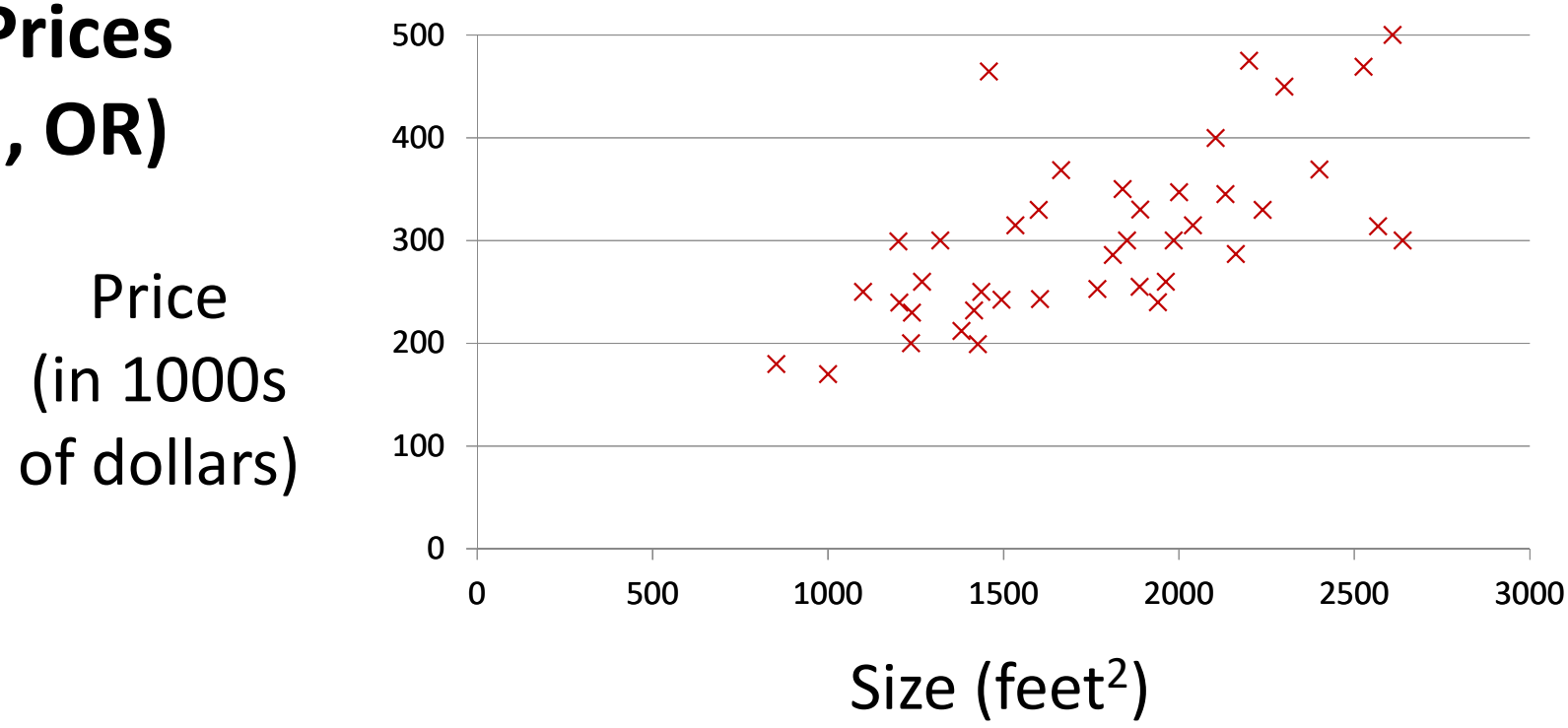


$$h_\theta(x) = \theta_0 + \theta_1 x$$

مدل — عرض از مبدا — شیب خط

تابع خطا: برای برآورد پارامترهای مناسب برای مدل، یک معیار اختلاف بین تخمین و مقدار واقعی y تعریف می‌کنیم.

Housing Prices (Portland, OR)



Supervised Learning

Given the "right answer" for each example in the data.

Regression Problem

Predict real-valued output

Simple Linear Regression

- θ_0 **and** θ_1 are known as the model coefficients or **parameters**.
- we have used our training data to produce estimates θ_0 and θ_1 for the model coefficients, we can predict future price on the basis of a particular value of size by computing

$$\hat{y} = \theta_0 + \theta_1 x$$

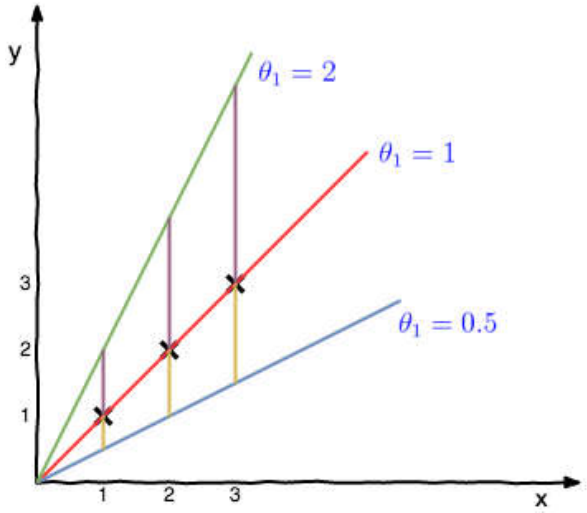
where $\hat{y}$ indicates a prediction of Y on the basis of $X = x$.

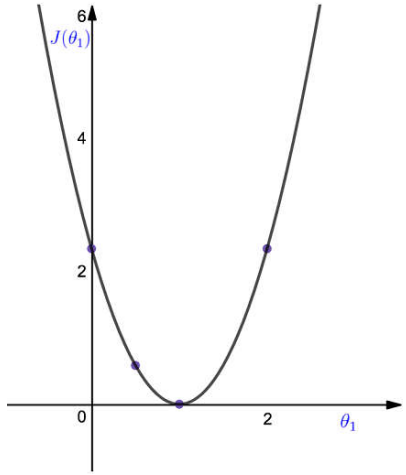
تابع هزینه یا خطا (Loss Function)

ایده این است که پارامترهای عرض از مبدا θ_0 و شیب خط θ_1 را طوری انتخاب کنیم که برای داده‌های آموزشی (x, y) مقدار خروجی y به $h_\theta(x)$ نزدیک باشد و هدف این است که تابع هزینه را با توجه به پارامترهای θ_0 و θ_1 کمینه کنیم.

تابع خطای میانگین مربعات (Mean Squared Error)

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$





حالت ساده با $\theta_0 = 0$

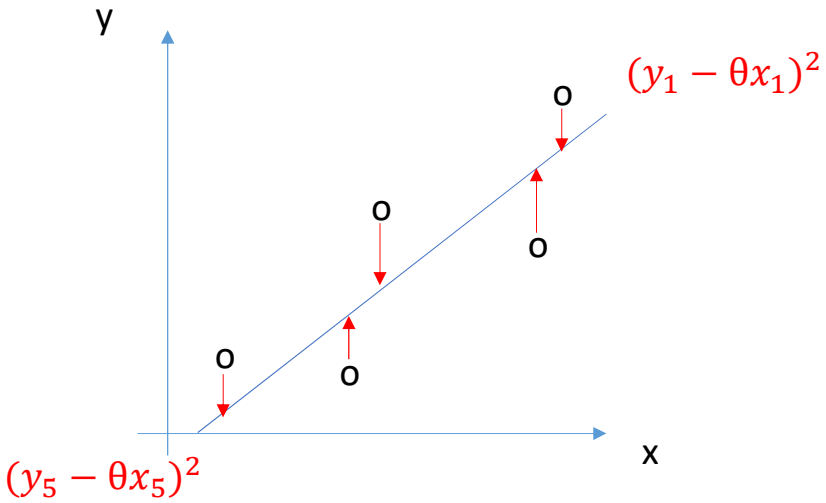
How do we learn the parameters θ?

We define the cost function:

$\underline{\theta} = [\theta_0, \theta_1]^T$

$$J(\underline{\theta}) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

We want to choose θ so as to minimize J(θ).



Optimization algorithm

• Using a search algorithm that starts with some "initial guess" for θ, and that repeatedly changes θ to make $J(\theta)$ smaller, until hopefully we converge to a value of θ that minimizes $J(\theta)$.

• Specifically, let's consider the **gradient descent algorithm**, which starts with some initial θ, and repeatedly performs the update:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

α is called the learning rate.

Gradient descent algorithm

repeat until convergence {

$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)

Simultaneous update

$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$

$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$

$\theta_0 := \text{temp0}$

$\theta_1 := \text{temp1}$

incorrect

$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$

$\theta_0 := \text{tempo}$

$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$

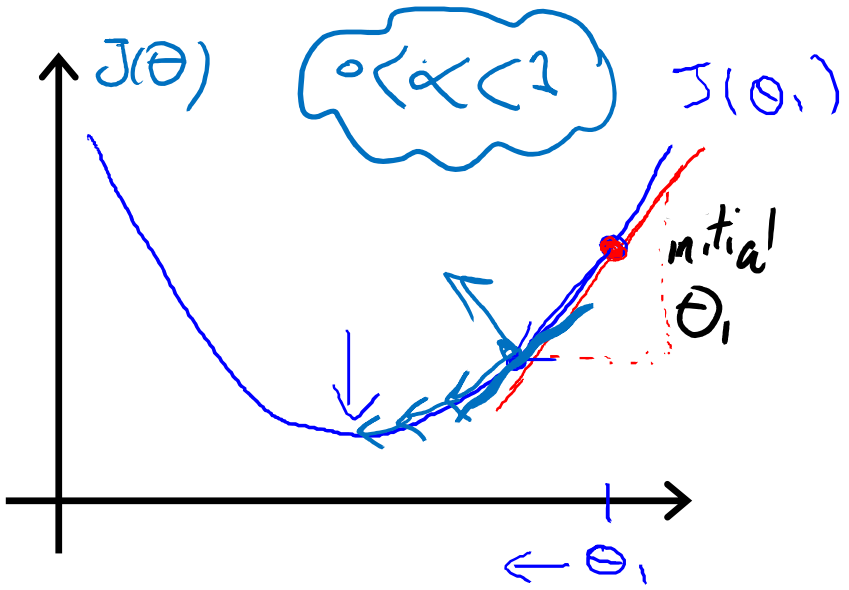
$\theta_1 := \text{temp1}$

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$

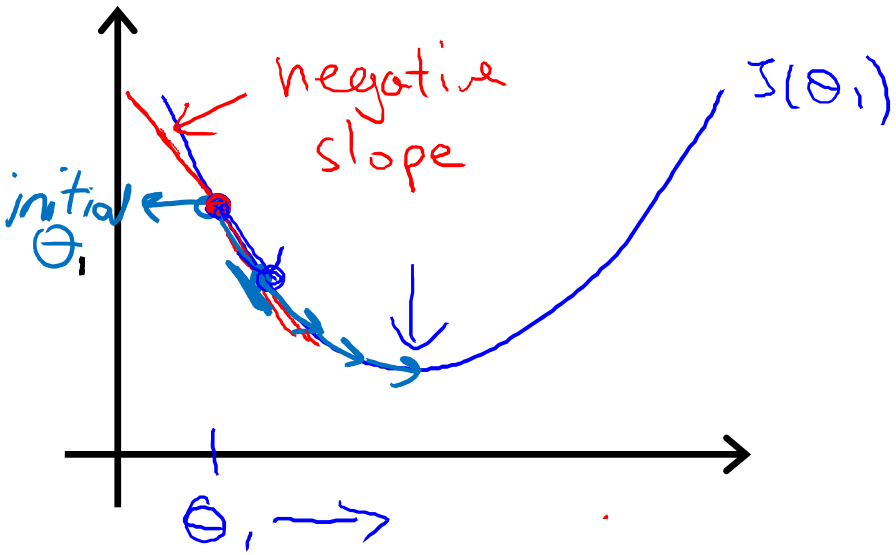


$(\theta_1 \in \mathbb{R})$

$$\theta_1 := \theta_1 - \alpha \underbrace{\frac{d}{d\theta_1} J(\theta_1)}_{\geq 0}$$

$\alpha \leftarrow$ $\frac{d}{d\theta_1}$

$$\theta_1 := \theta_1 - \alpha \quad (\text{positive number})$$



$$\underbrace{\frac{d}{d\theta_1} J(\theta_1)}_{\leq 0}$$

$$\theta_1 := \theta_1 - \alpha \quad (\text{negative number})$$

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$$= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2$$

$j = 0, 1$

$h_\theta(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$

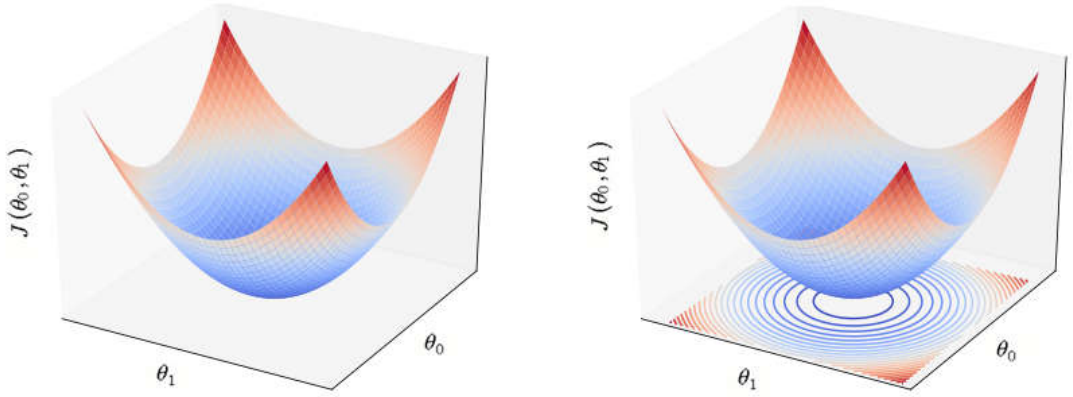
$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)}$$

تابع هزینه یا خطا (Loss Function)

نمایش کانتور

نمایش سه بعدی



گرادیان کاهشی (Gradient Descent)

نرخ یادگیری (Learning Rate) ≤ صفر

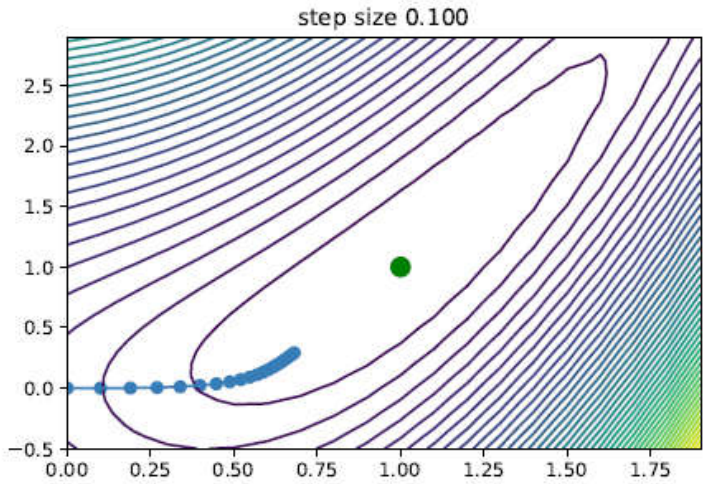
$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}, \qquad j = 0, 1$$

مشتق

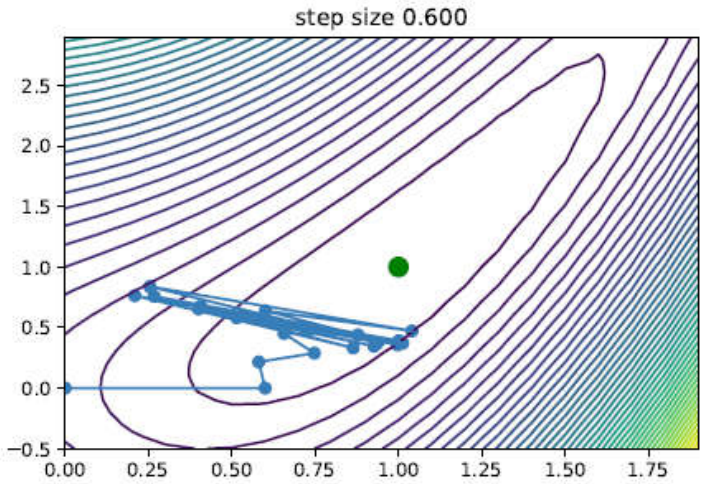
$$\begin{cases} \theta_0 := \theta_0 - \dfrac{\alpha}{m} \displaystyle\sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)}) \\ \theta_1 := \theta_1 - \dfrac{\alpha}{m} \displaystyle\sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)}) x^{(i)} \end{cases}$$

Step size (learning rate)

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$



(a)



(b)

The global minimum is at (1;1)

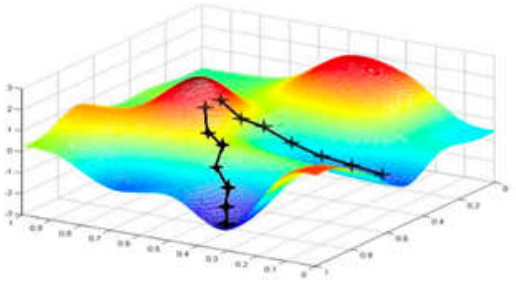
(a) 0/1

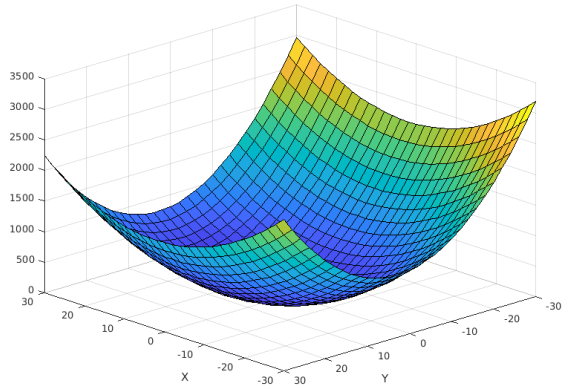
move slowly along the valley.

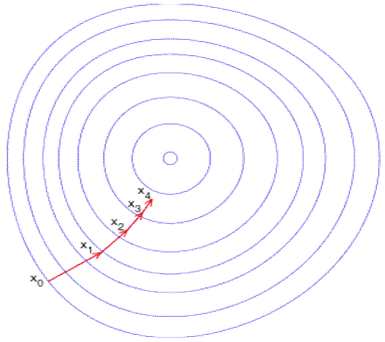
(b) 0/6

start oscillating up and down the sides of the valley and never converge to the optimum, even though this is a convex problem.

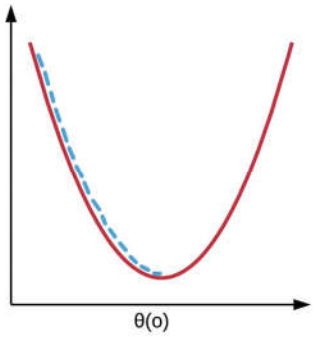
تابع محدب (Convex) و غیرمحدب (Non-convex)

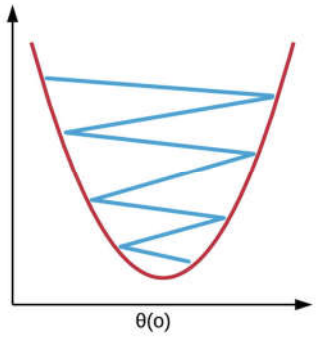




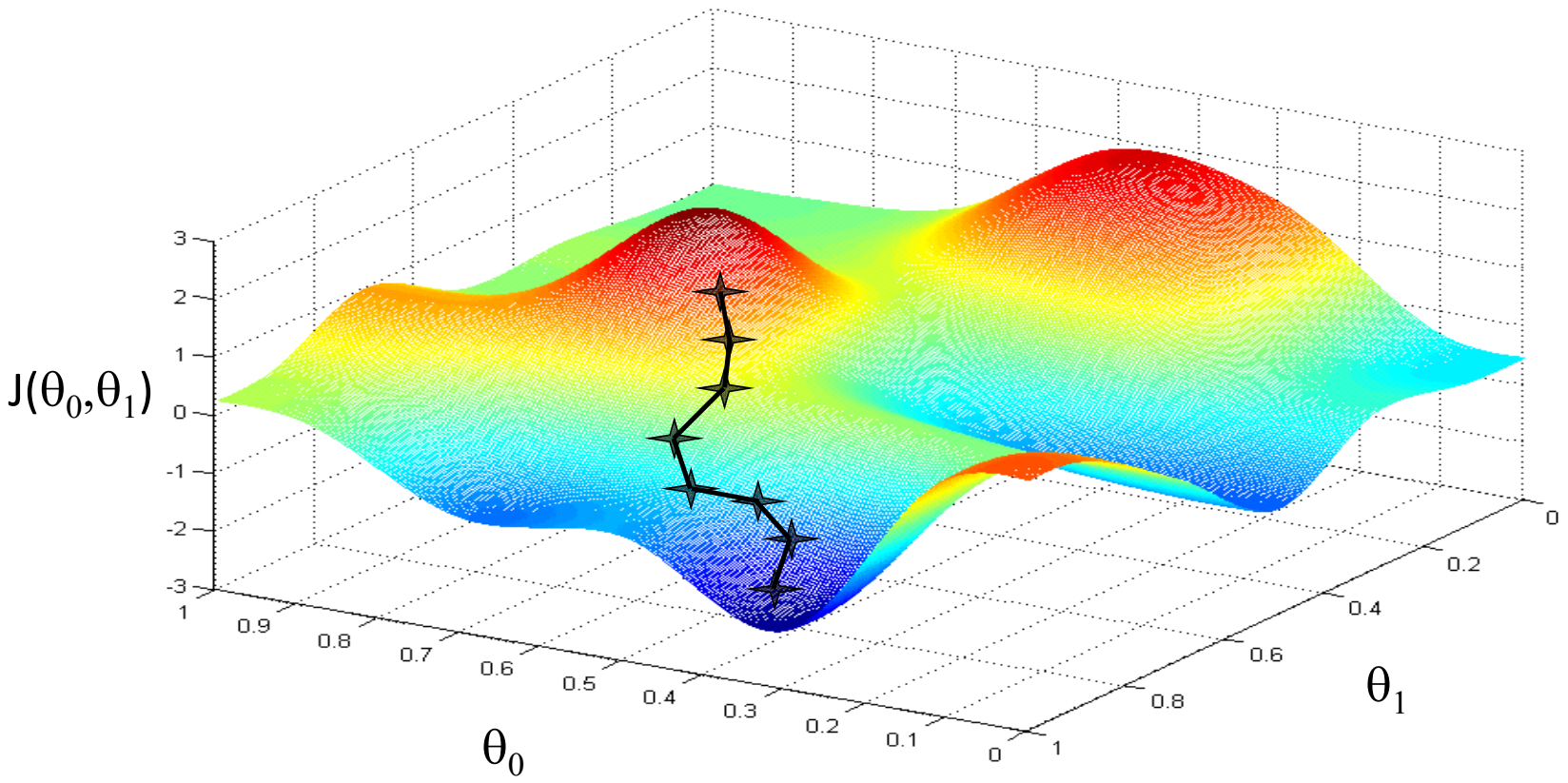


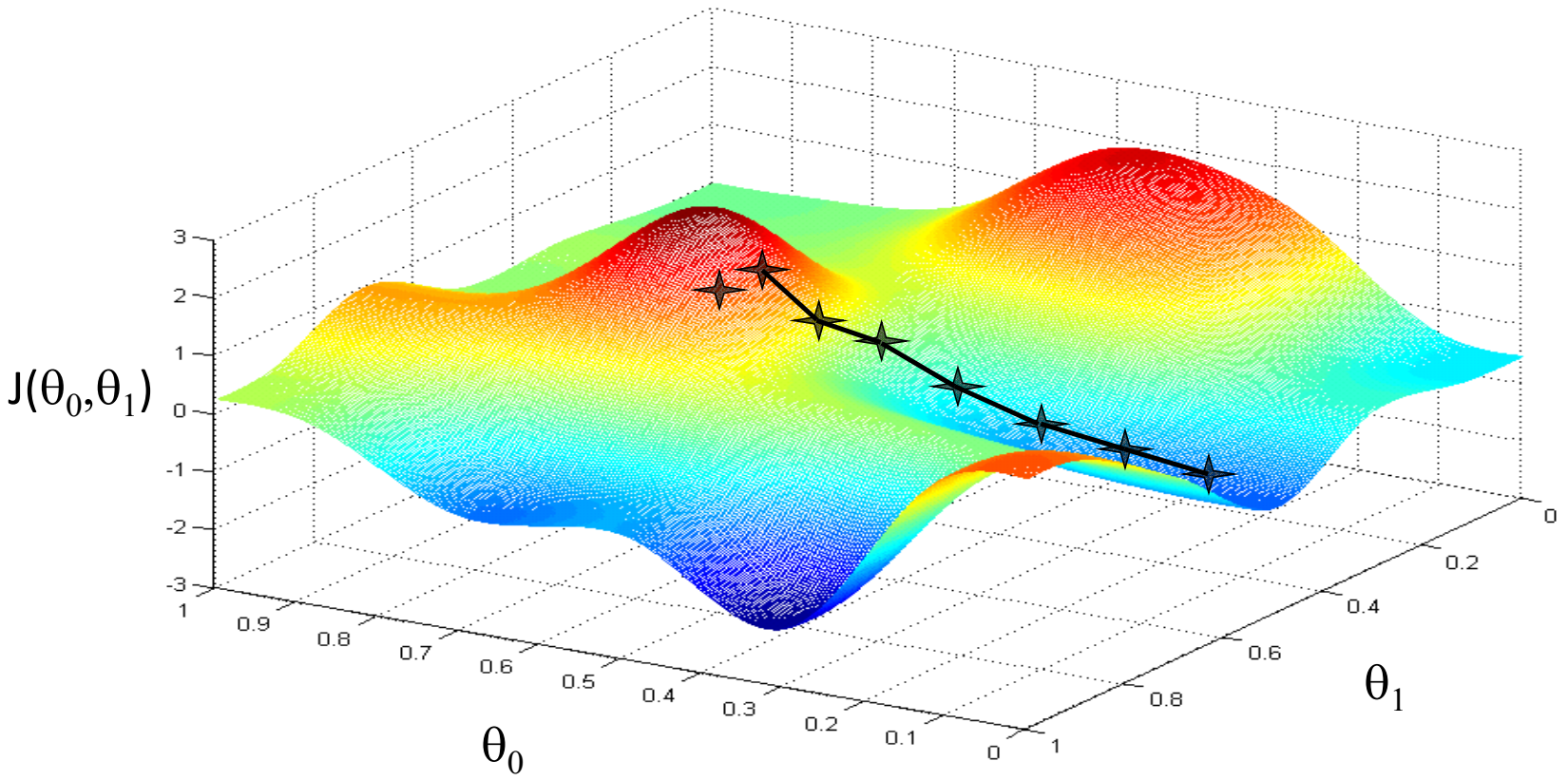
تاثیر نرخ یادگیری

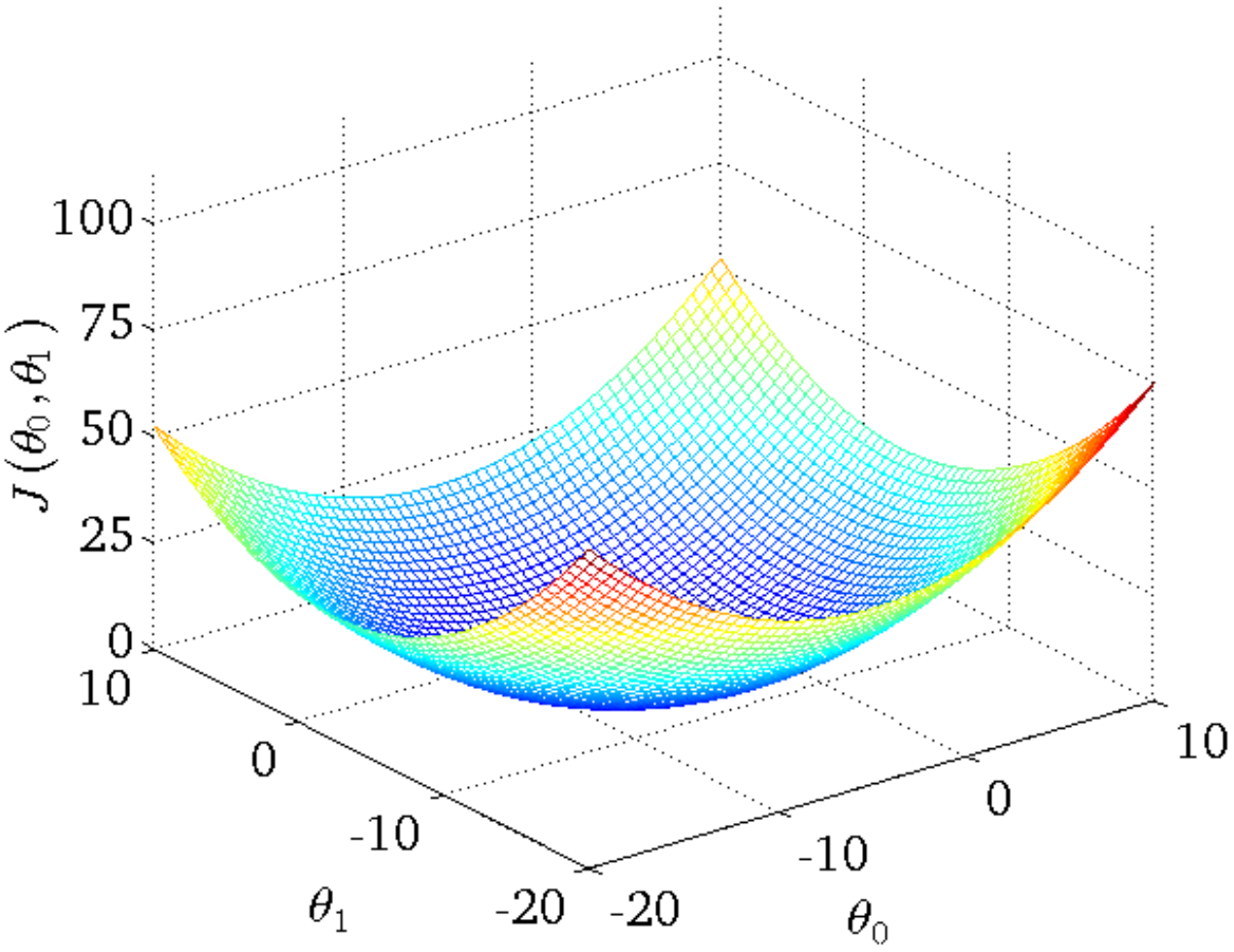




- نرخ یادگیری زیاد
- نرخ یادگیری کم

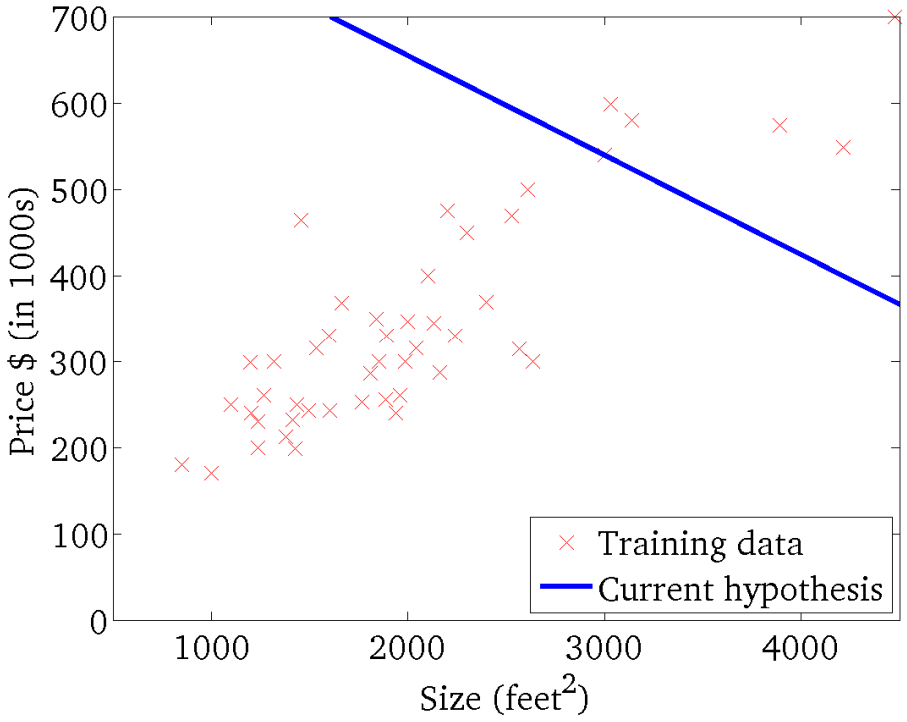

$J(\theta_0,\theta_1)$
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ_0
0
0.2
0.4
0.6
0.8
1
θ_1


$J(\theta_0,\theta_1)$
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ_0
0
0.2
0.4
0.6
0.8
1
θ_1


100
75
50
25
0
$J(\theta_0, \theta_1)$
10
0
-10
-20
θ_1
-20
-10
0
10
θ_0

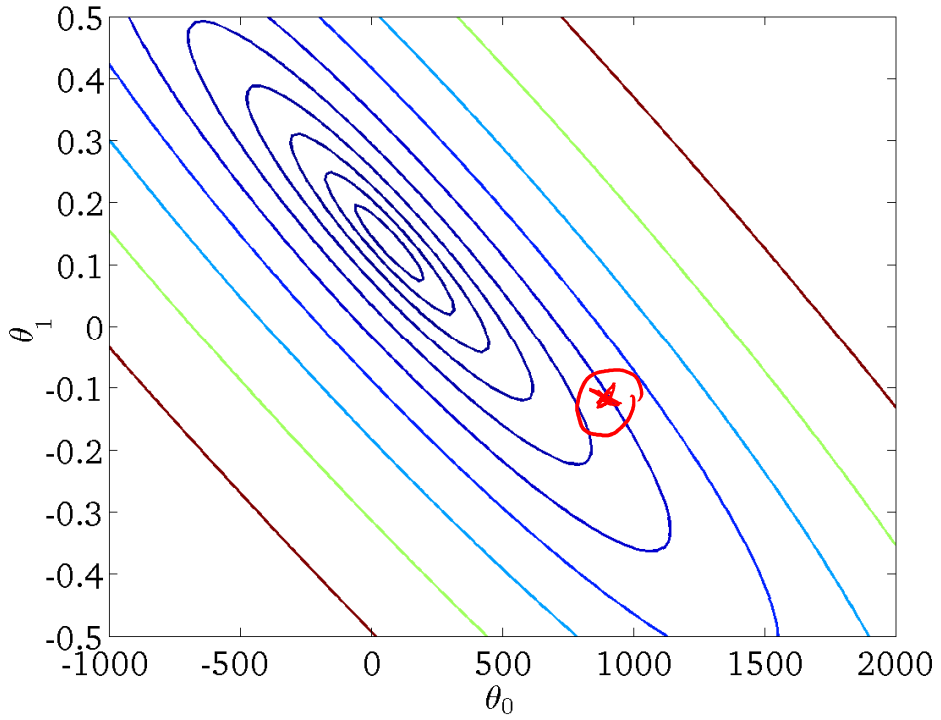
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



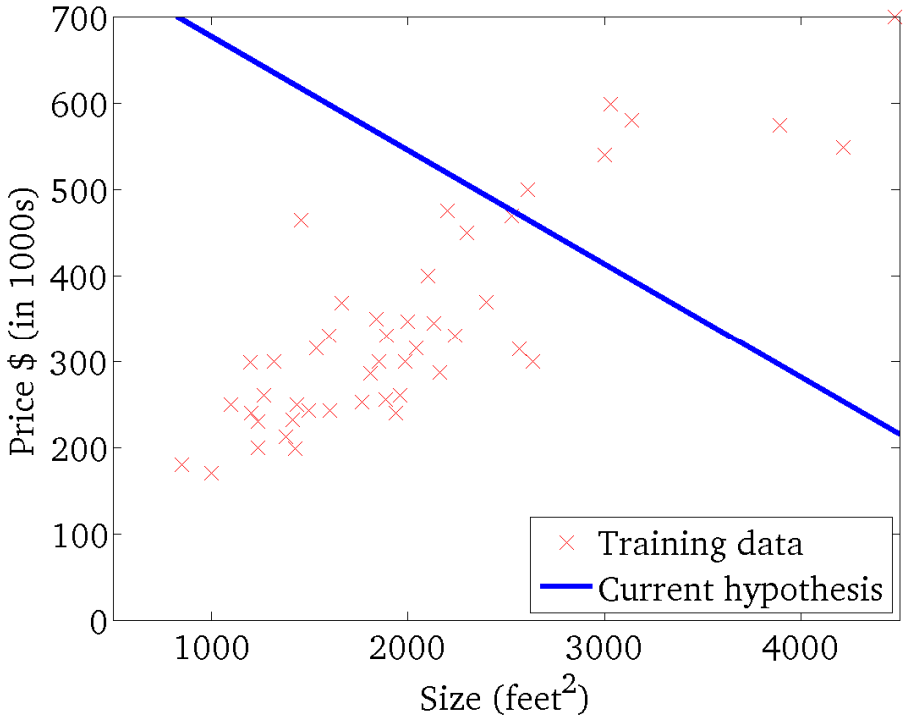
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



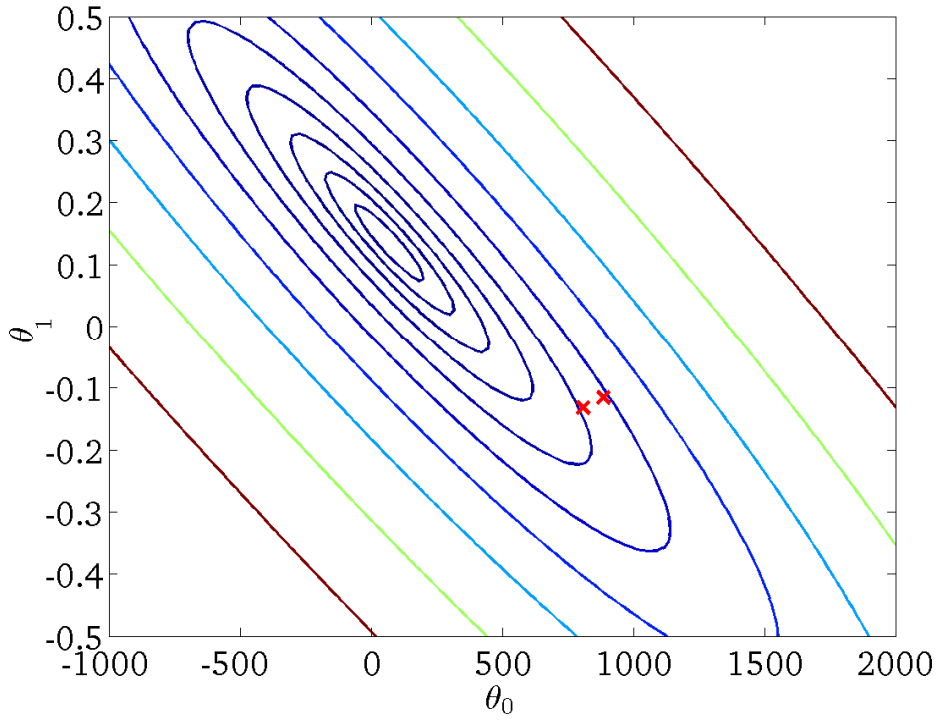
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



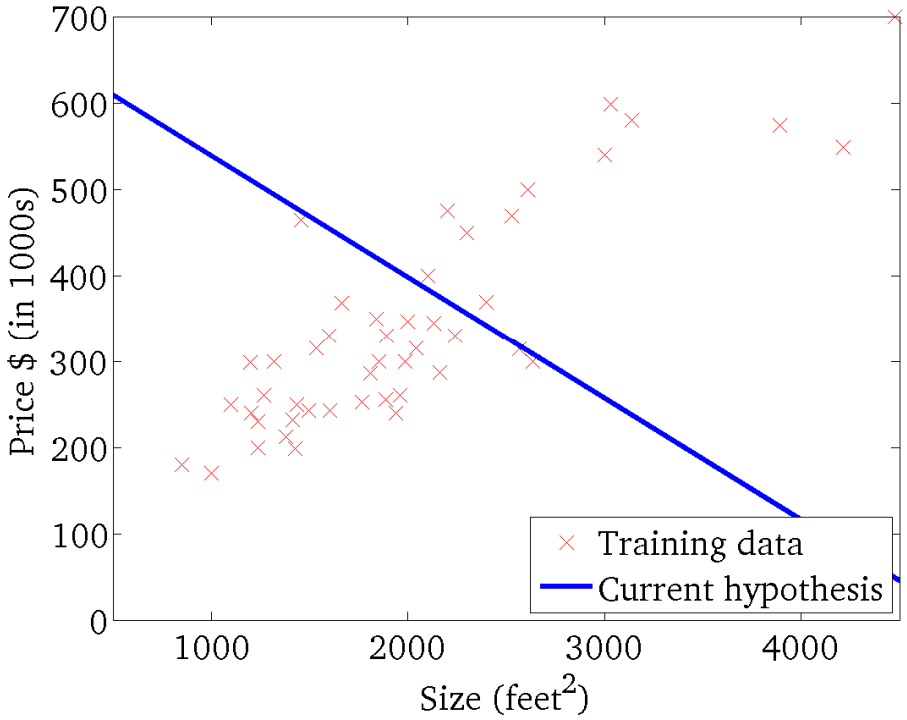
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



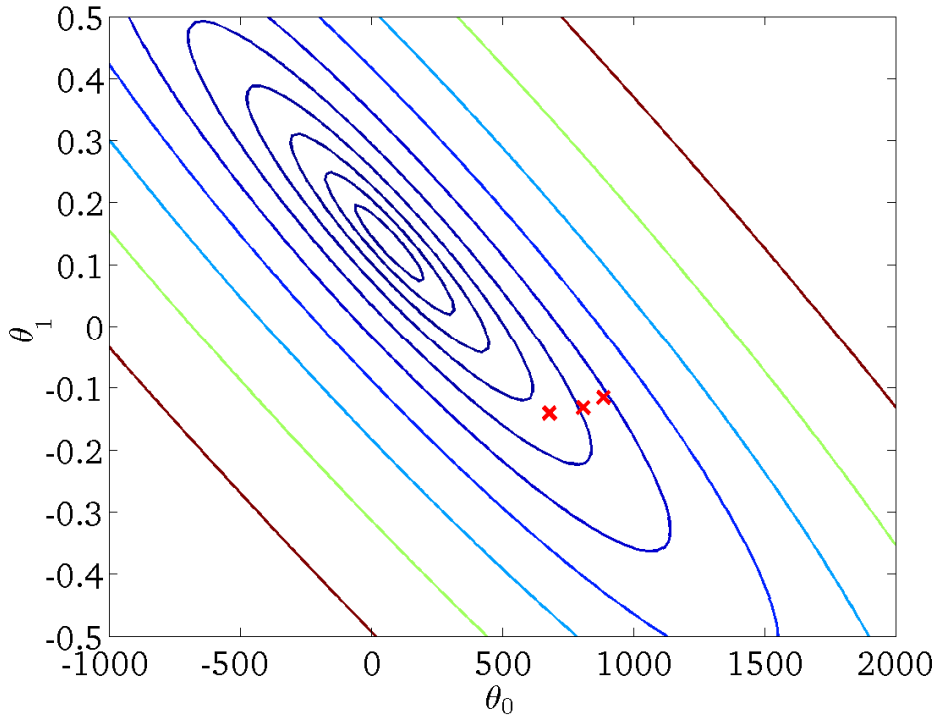
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



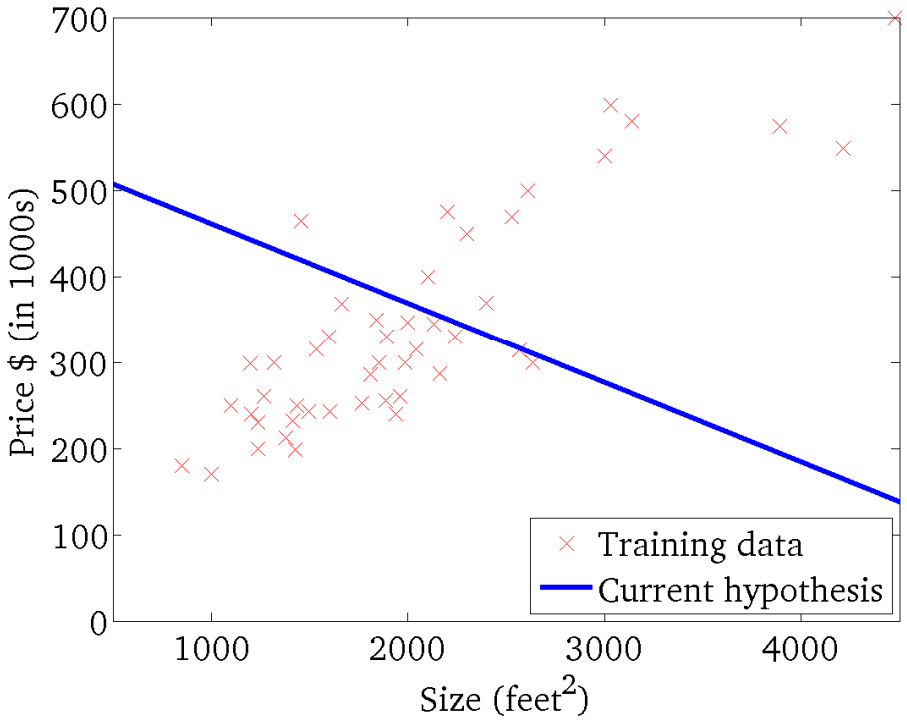
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



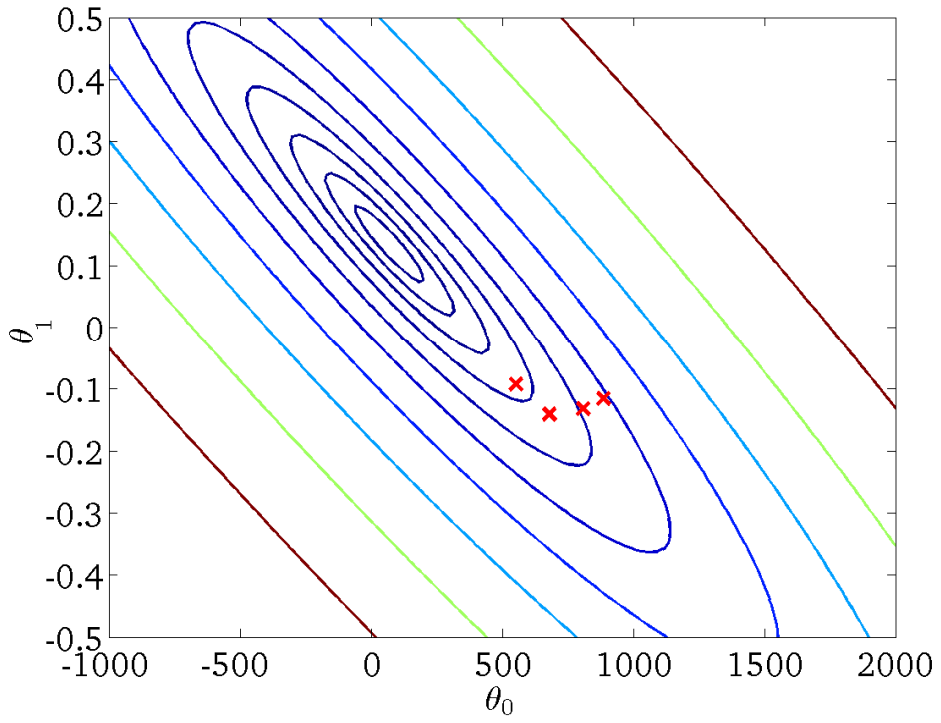
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



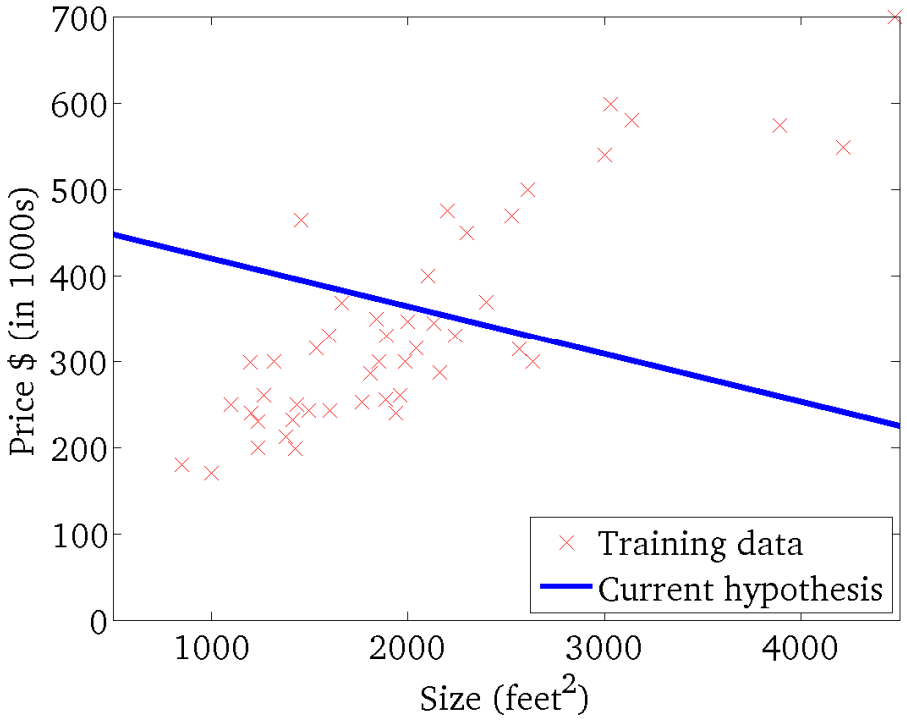
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



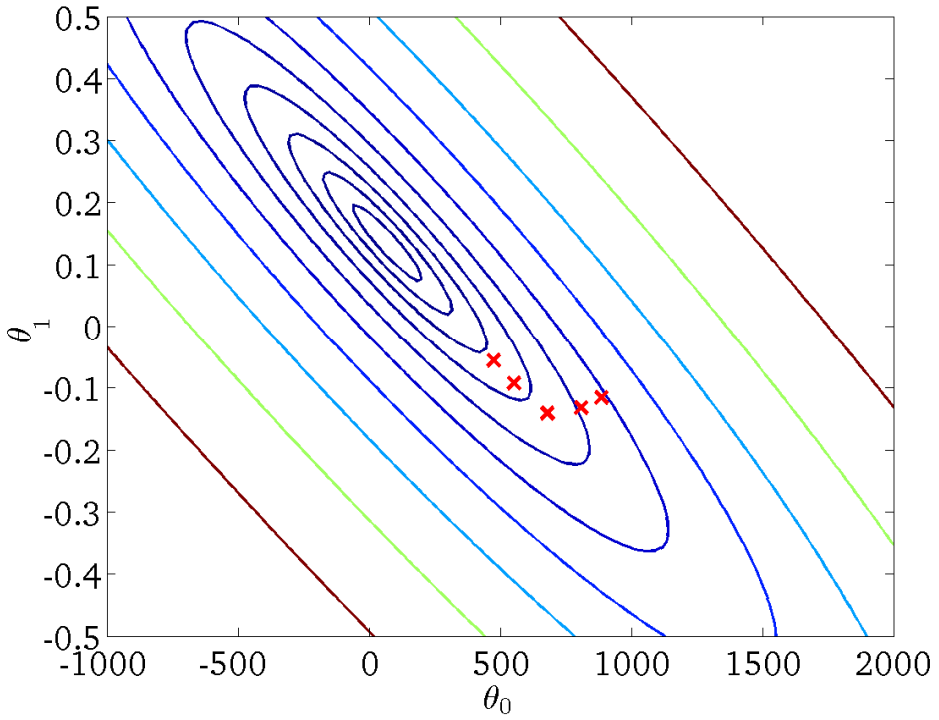
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



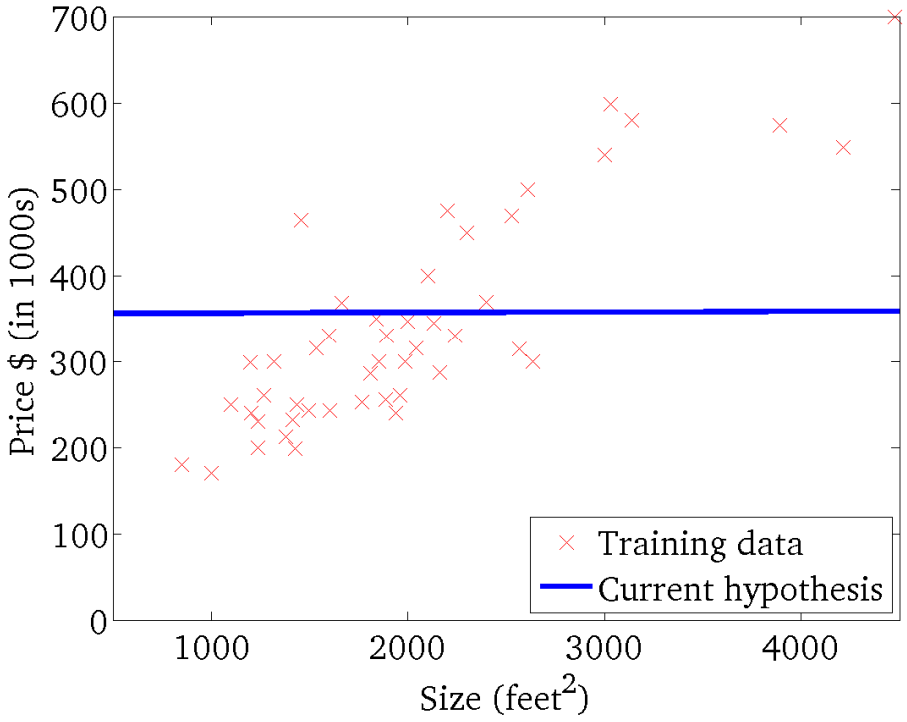
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



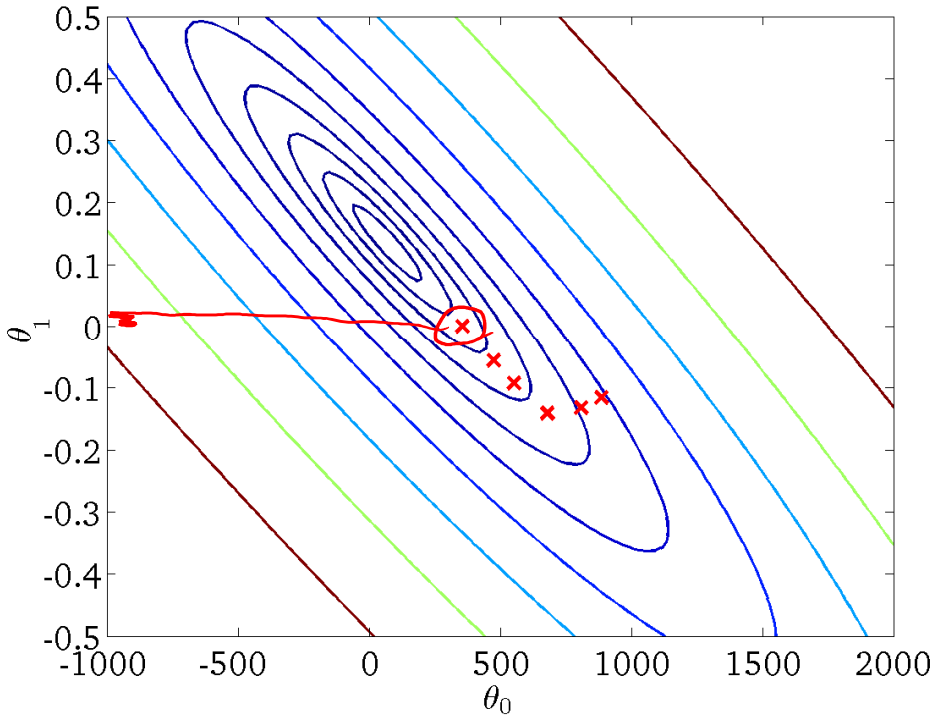
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



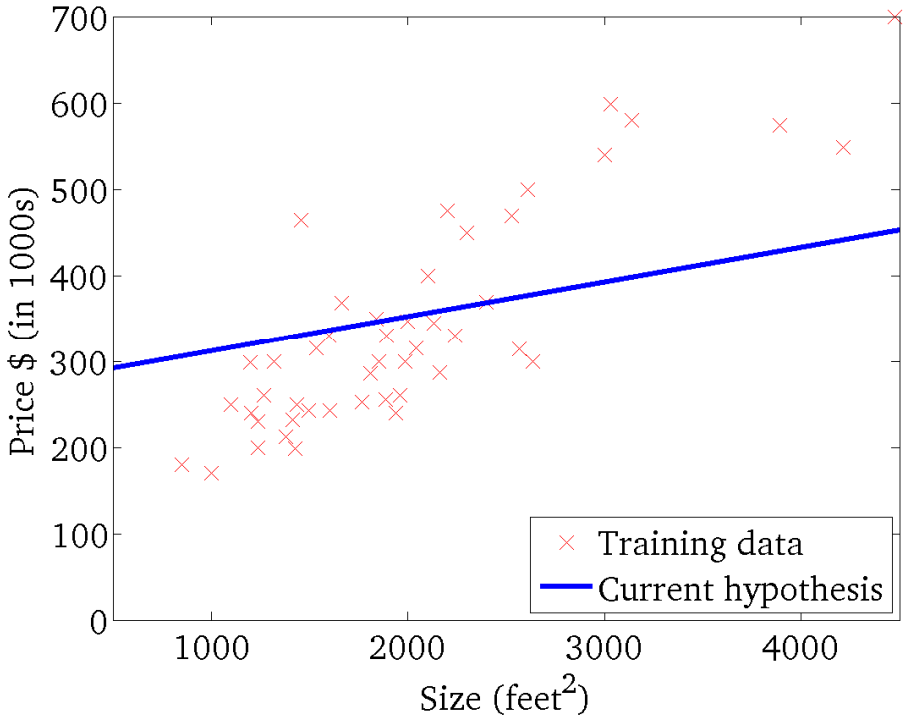
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



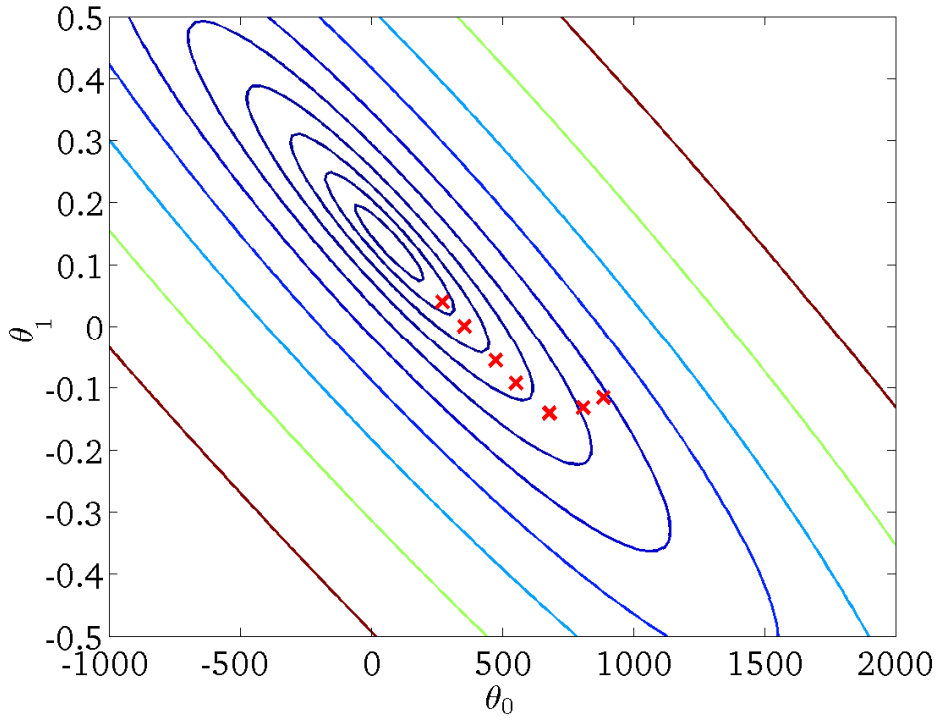
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



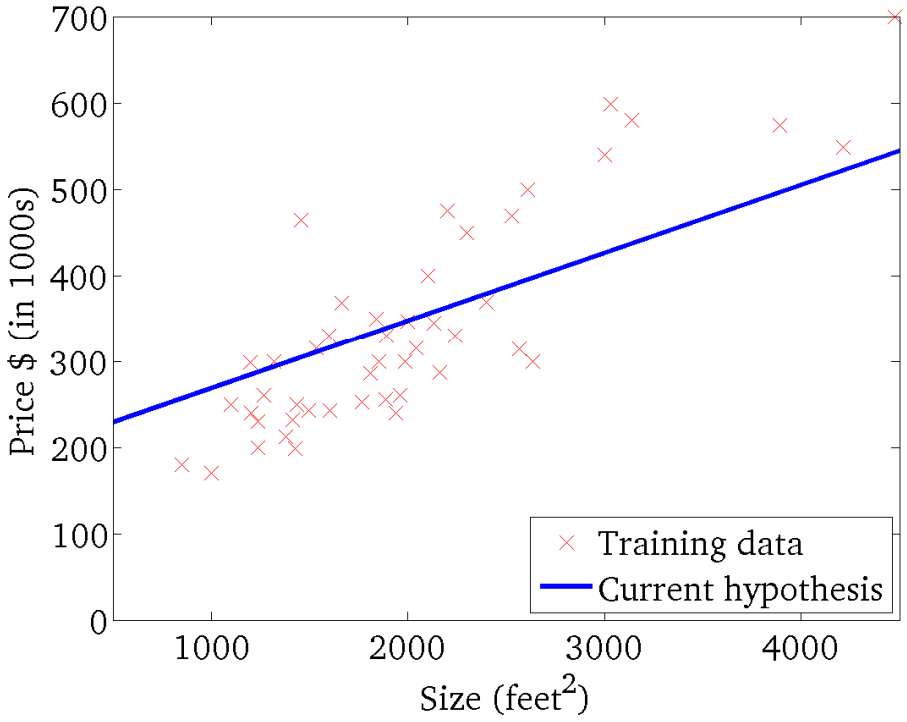
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



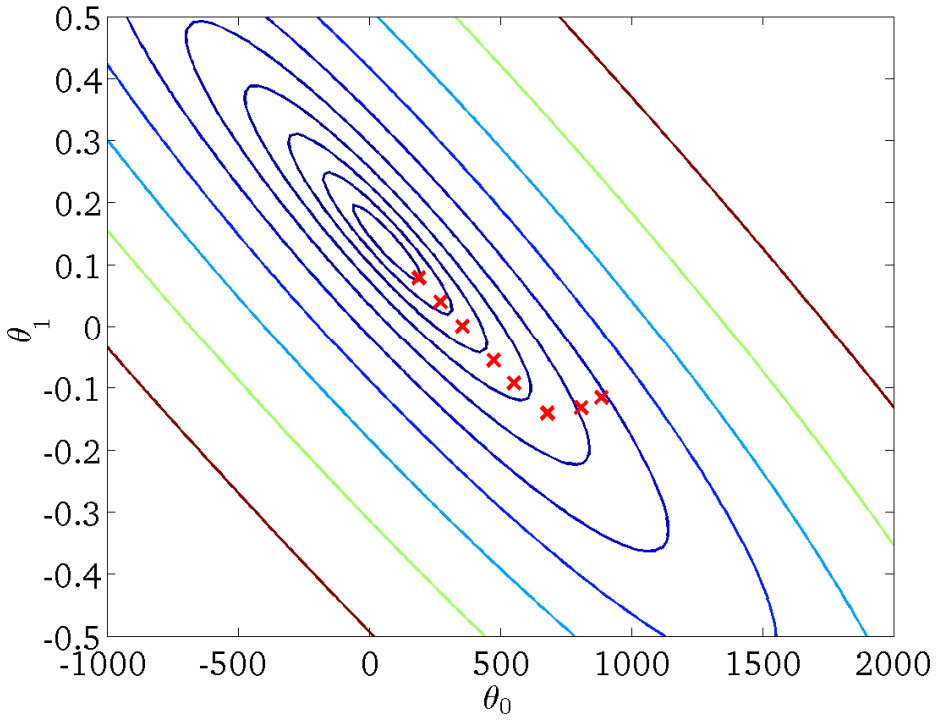
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



برازش خطی چندمتغیره

❖ **برازش خطی چندگانه** (Multiple Linear Regression) برای پیشبینی یک متغیر وابسته (y) که به صورت خطی با بیش از یک متغیر مستقل ($x_1, x_2, \dots, x_n$) مرتبط است به کار می‌رود.

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \theta^T X$$

$x_0 = 1$

$x_j^{(i)}$: مقدار ویژگی j در نمونه i

$x^{(i)}$: ویژگی‌های نمونه i که می‌توانیم آن‌ها را به شکل بردار $X = [1, x_1, x_2, \dots, x_n] \in \mathbb{R}^{n+1}$ تعریف کنیم. $x_0^{(i)} = 1$ است.

θ: بردار ضرایب که به شکل $\theta = [\theta_0, \theta_1, \theta_2, \dots, \theta_n] \in \mathbb{R}^{n+1}$ آن را تعریف می‌کنیم.

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} = \theta_j - \frac{\alpha}{m} \sum_{i=1}^{m} (h_\theta(x)^{(i)} - y^{(i)}) x_j^{(i)}$$

Multiple features

Notation:

n= number of features

$x^{(i)}$ = input (features) of ith training example.

$x_j^{(i)}$ = value of feature j in i^{th} training example.

x_1 Size (feet²)	x_2 Number of bedrooms	x_3 Number of floors	x_4 Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

Size (feet²) x	Price ($1000) y
2104	460
1416	232
1534	315
852	178
…	…

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Multiple features

Hypothesis

Previously: $h_\theta(x) = \theta_0 + \theta_1 x$

Multivariate linear regression: $h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

x_1	x_2	x_3	x_4	y
Size (feet²)	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4$$

Gradient descent for multiple variables

Hypothesis: $h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

MSE

Gradient descent:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

$\}$

(simultaneously update for every $j = 0, \ldots, n$)

$\theta := \theta - \alpha \nabla J$

$\theta_0 \ldots \theta_n \quad \nabla J = \left(\frac{\partial J}{\partial \theta_0}, \frac{\partial J}{\partial \theta_1}, \ldots, \frac{\partial J}{\partial \theta_n} \right)$

Gradient Descent

Previously (n=1):

Repeat{

$$\theta_0 := \theta_0 - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm $(n \geq 1)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

}

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

$\ldots$

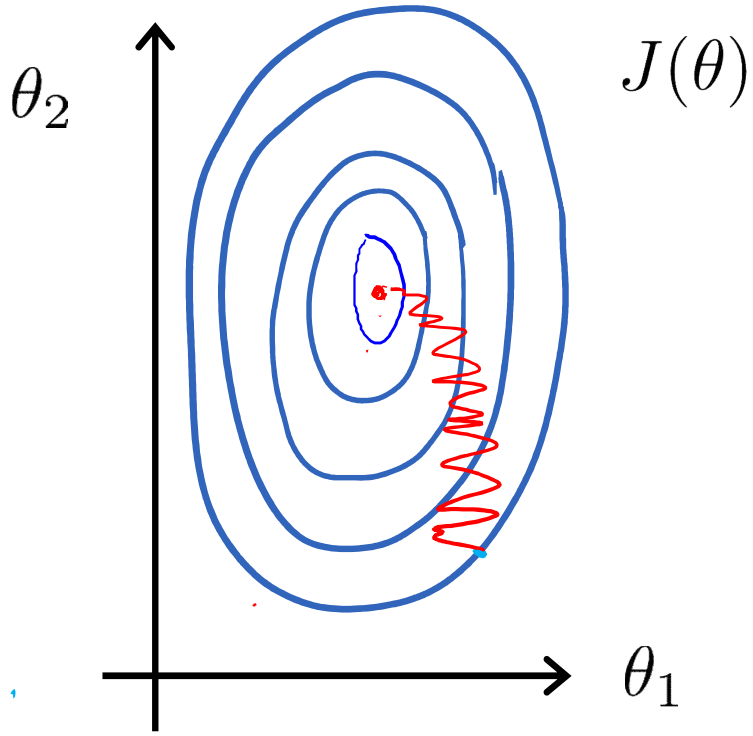
$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Feature Scaling

Idea: Make sure features are on a similar scale.

E.g. x_1 = size (0-2000 feet^2)

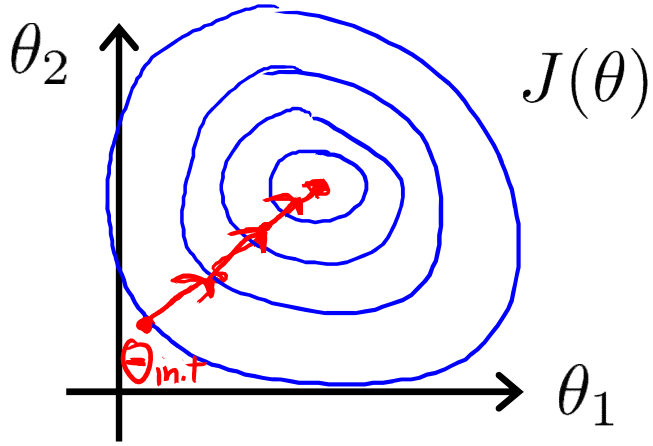
x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

$$x_2 = \frac{\text{number of bedrooms}}{5}$$

$0 \leq x_1 \leq 1$ $\quad$ $0 \leq x_2 \leq 1$



Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean (Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{size - 1000}{2000}$

$x_2 = \frac{\#bedrooms - 2}{5}$

$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$

$x_1 \leftarrow \frac{x_1 - \mu_1}{s_1}$ | $x_2 \leftarrow \frac{x_2 - \mu_2}{s_2}$

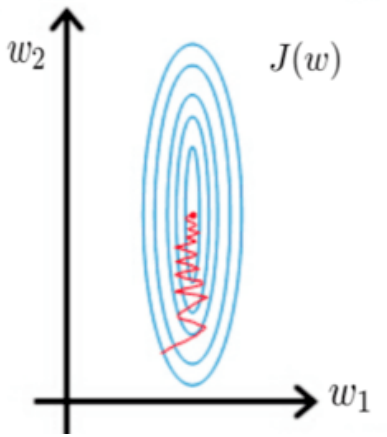
μ_1: avg value of x_1 in training set

s_1: range (max - min) (or standard deviation)

تغییر مقیاس متغیرها

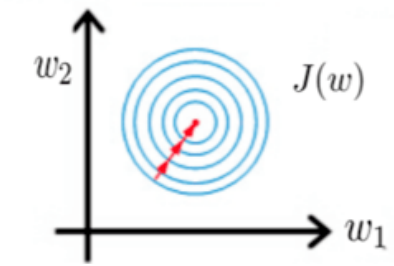
Gradient descent without scaling

$x_1 >> x_2$



Gradient descent after scaling variables

$0 \leq x_1 \leq 1$
$0 \leq x_2 \leq 1$



min-max normalization

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Mean normalization

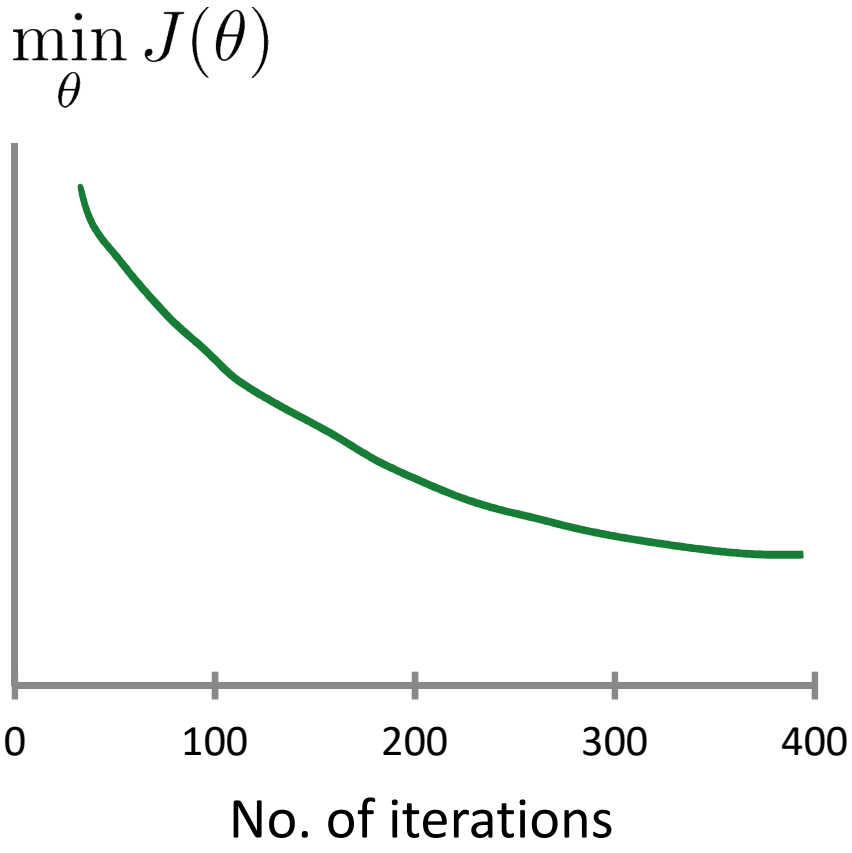
$$x' = \frac{x - mean(x)}{max(x) - min(x)}$$

Gradient descent

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- "Debugging": How to make sure gradient descent is working correctly.
- How to choose learning rate α.

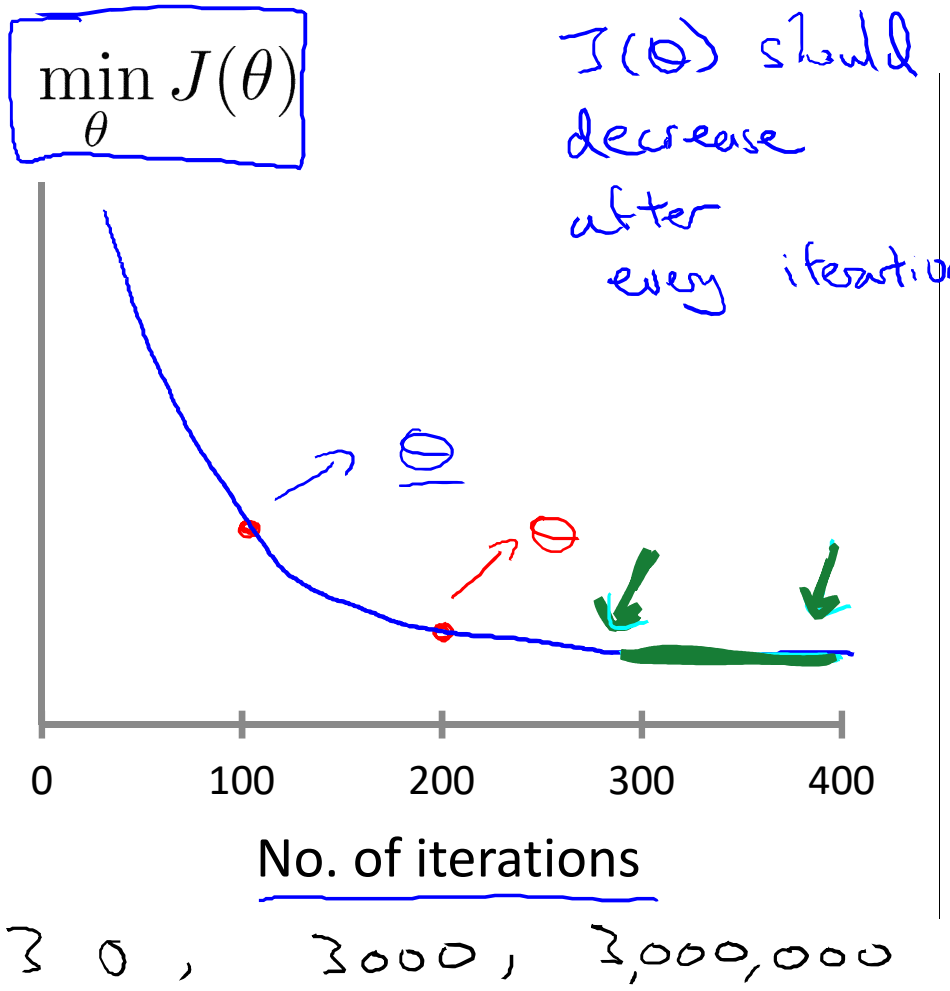
Making sure gradient descent is working correctly.



Example automatic convergence test:

Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

Making sure gradient descent is working correctly.

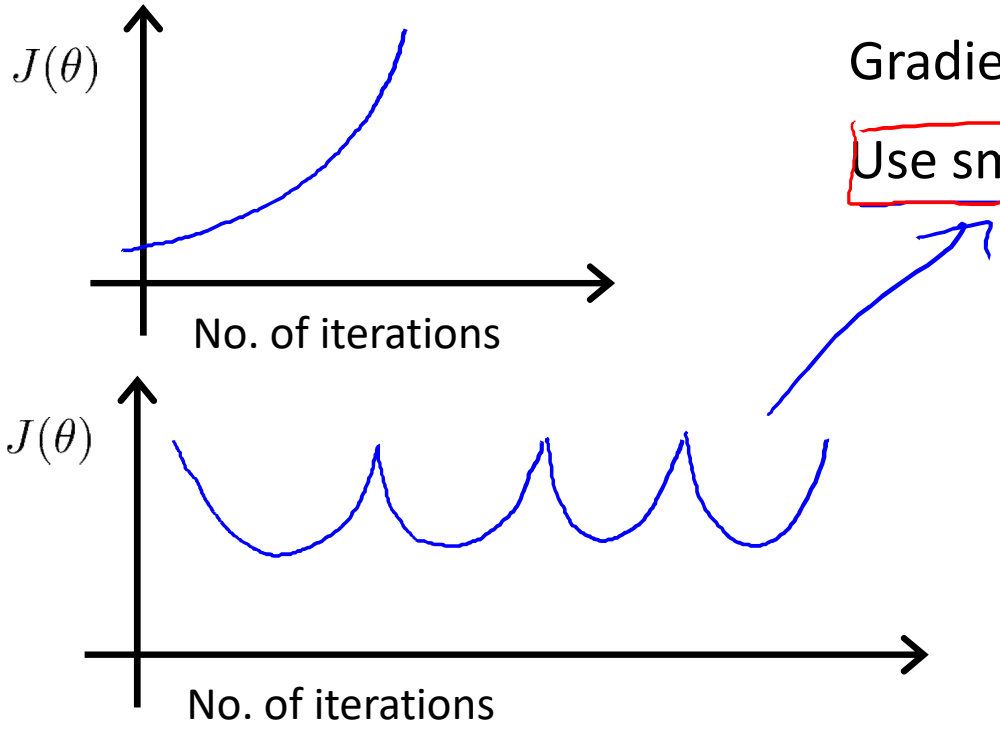


→ Example automatic convergence test:

→ Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

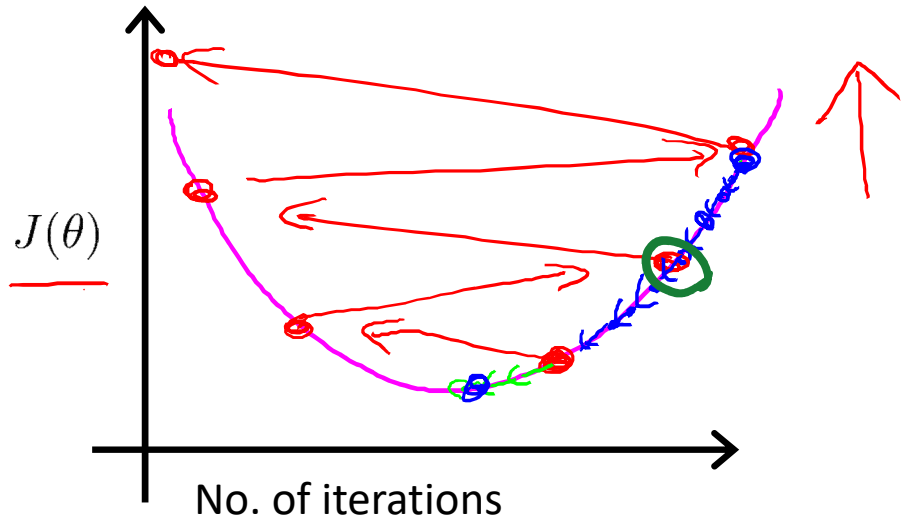
ϵ

Making sure gradient descent is working correctly.



Gradient descent not working.

Use smaller α



- For sufficiently small α, $J(\theta)$ should decrease on every iteration.
- But if α is too small, gradient descent can be slow to converge.

Summary:

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.



(Slow converge also possible.)

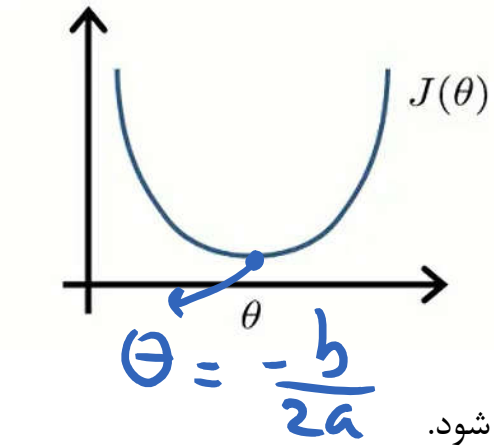
To choose α, try

$$\ldots, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, \ldots$$



روش آماری Normal Equation

$$J(\theta) = a\theta^2 + b\theta + c$$



حالت ساده به‌دست آوردن کمینه یک تابع:

$$\frac{dJ(\theta)}{d\theta} = 0 \rightarrow \Theta = -\frac{b}{2a}$$

اما در مسئله ما $\theta \in \mathbb{R}^{n+1}$ است و مشتق باید برای همه θ ها برابر صفر قرار بگیرد و حل شود.

$$\frac{\partial J(\theta)}{\partial \theta_j} = 0 \ (for\ every\ j) \rightarrow \theta_0, \dots, \theta_n$$

$j = 0 \dots n$

Normal equation

(رگرسیون خطی)

$\theta \in \mathbb{R}^{n+1} \Rightarrow J(\theta_0, \theta_1, \ldots, \theta_n)$ $\frac{\partial J}{\partial \theta_j} = 0 \Rightarrow$ $j = 0 \ldots n$

بیشینه $\theta_0, \ldots, \theta_n$

Normal equation: Analytic solution

$$X = \begin{bmatrix} X_0 & X_1 & X_2 & X_3 & X_4 \\ 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$$

$$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

$$X = \begin{bmatrix} (x^{(1)})^T \\ (x^{(m)})^T \end{bmatrix}_{m \times (n+1)}$$

$$y = \begin{bmatrix} y^{(1)} \\ y^{(m)} \end{bmatrix}_{m \times 1}$$

$$\theta = (X^T X)^{-1} X^T y$$

$(X^T X)^{-1}$ is inverse of matrix $X^T X$

m **training examples,** n **features.**

* نیاز به مقیاس کردن ویژگی‌ها لازم نیست

Gradient Descent

- Need to choose α.
- Needs many iterations.
- Works well even when n is large.

$n = 10^6$

Normal Equation

- No need to choose α.
- Don't need to iterate.
- Need to compute $(X^T X)^{-1}$ → $O(n^3)$
- Slow if n is very large.

$n = 100$

$n = 1000$

What if $X^T X$ is non-invertible? (singular/ degenerate)

- Redundant features (linearly dependent).
  E.g. x_1 size in feet^2
  x_2 size in m^2

$\Rightarrow X^T X$ singular

- Too many features (e.g. $m \leq n$).
  - Delete some features, or use regularization.

$n = 100$ $m = 10$

θ: 101×1